Clean Code & C# Standards

Quick one-look revision notes | 4 modules, 11 topics

1 Intro	2 Naming	3 Methods	4 Guards	5 Readability	6 House Casing
7 Automation	8 PR Review	9 Async	10 Feedback	11 Gate	

1 Intro: Why Clean Code?

- **Readable** = names show what a thing holds, functions do one job, logic is plain. **Maintainable** = next person can change it safely.
- Standards are a **shared team responsibility**, not personal taste. Same naming, folders and error handling make code predictable and cut onboarding time.
- Course goal: write PR-friendly code, review others kindly, and use a checklist.
- Example: one 200-line `ProcessData` becomes `ValidateInput`, `TransformRecords`, `SaveToDatabase` - bugs found in minutes, not hours.

2 Naming & Magic Values

Intent-revealing names

- Vague names (`x, d, i, tmp, s, data`) force readers to decode. Name for **what it holds**, not type or position.
- A good name is **living documentation**: comments drift, names can't lie. Want to add a comment? **Rename instead.**
- Methods say what they decide/do: `IsEligibleForDiscount()`, `ValidatePayment()` - not `CheckStatus()` or `Check()`.
- Write abbreviations in full: `emp, cnt, flg` become `employeeName, itemCount, isProcessingComplete`.
- Ask: *"What does this variable represent?"* If you can't answer in one short phrase, the name is too vague.

BAD

```
int d = 7;
int x = 0;
foreach (var item in orders)
    x += item.Price;
```

GOOD

```
int daysUntilExpiry = 7;
int totalOrderAmount = 0;
foreach (var order in orders)
    totalOrderAmount += order.Price;
```

Magic numbers & strings

- Hard-coded literals with no explanation: `0.15, 401, 86400, "PENDING"`.
- Extract to **named constants** for: **clarity** (business rule is named), **single source of truth** (change once), **searchability**.
- Name the **purpose**, not the type: `MAJOR_DISCOUNT_THRESHOLD` beats `DISCOUNT_LIMIT`; `SECONDS_PER_HOUR` beats `MAGIC_NUMBER_3600`.
- Steps: spot value → decide what it means → pick a purposeful name → declare const & replace everywhere.
- Don't over-extract: a number that is obvious in context and used once can stay inline.
- `const` = fixed at compile time. `static readonly` = value known only at runtime.

BAD

```
decimal tax = orderTotal * 0.15m;
if (age > 65) { Retire(); }
```

GOOD

```
const decimal StandardTaxRate = 0.15m;
const int RETIREMENT_AGE = 65;
decimal tax = orderTotal * StandardTaxRate;
if (age > RETIREMENT_AGE) { Retire(); }
```

Remember: In PR review, asking for a rename is a fair improvement, not a nitpick.

3 Small Methods & Fewer Parameters

- **Too many parameters** (7+) = hard to read, painful to test (set up all of them), and costly to change (every caller breaks).
- It often signals the method does too much, or related data isn't grouped.
- **Parameter object**: a class/record holding related fields. Group by **shared meaning**, not list position. If you can't name the group, it's not a real group.

BAD

```
void CreateUser(string firstName,
  string lastName, string email,
  string password, string phone,
  string address, string city,
  string state, string zip)
```

GOOD

```
void CreateUser(PersonalInfo p,
  Credentials c, Address a,
  string phone)
// 9 params -> 4, each one concept
```

- Refactor steps: **identify groups** → **create classes (Extract Class / Introduce Parameter Object)** → **update signature**. Logic and results stay identical.
- **Defaults & overloads**: `bool notifyUser = true`. Keep to 1-2 overloads. Don't hide *required* params behind defaults.
- Adding a field later (e.g. middle name) changes the class, **not** the method signature.

Grouping vs. method redesign

Situation	Better fit
Params are related, method does one job well	**Parameter grouping**
Method has multiple distinct responsibilities	**Method redesign** (split into smaller methods)
Unsure where responsibility lies	Start with redesign; grouping follows naturally

- Ask in order: (1) one job or several? (2) are params semantically grouped? (3) would smaller methods be easier to test/reuse?
- Metrics that improve: **parameter count** falls, **cognitive complexity** drops (extract focused methods), **testability** rises (fewer dependencies/mocks).

4 Guard Clauses & Early Returns

- Deep nested `if-else` = staircase; real work buried; brain tracks many conditions.
- **Guard clause** = check at the top of a method that returns/throws if a **precondition** fails. The **happy path** stays flat and unindented.
- Write `if (order == null) return;` (guard what *stops* you), not `if (order != null) { ... }`.

BAD

```
if (order != null) {
  if (order.Items.Count > 0) {
    if (order.Total > 0) {
      SaveOrder(order);
    }
  }
}
```

GOOD

```
if (order == null) return;
if (order.Items.Count == 0) return;
if (order.Total <= 0) return;

SaveOrder(order);
```

3-step refactor

- **1.** Identify preconditions in each nested if. **2.** Invert the condition and move it to the top. **3.** Unindent the remaining code.

Pick the right exit

Exit	Use when
`return`	Early exit, simple failure value
`throw`	Invalid state/argument (e.g. ArgumentNullException)
`continue`	Skip an item inside a loop

Quality	Nested if	Guard clause
Indentation	Grows per check	Stays flat
Main logic	Buried at bottom	Top level

Quality	Nested if	Guard clause
New check	Adds a nesting layer	Adds one guard
Cognitive load	Track many open blocks	Check and move on

- One precondition per guard, one line each. Order **cheapest first** (null/bool flags), expensive lookups last.
- Complex condition? Extract to a helper like `IsOrderValid()`.
- Caveat: if different failures need different behavior, keep one level of nesting or use another pattern.

5 Readability Over Cleverness (LINQ)

- Clever one-liners save lines but cost teammates minutes each read. **Cognitive load** = mental effort to understand code.
- Cryptic names in LINQ (`o.S, o.T, o.D`) turn reading into archaeology.
- Fix: break the chain into **named steps**, use real property names, extract constants (`ACTIVE_STATUS`).
- Maintainability beats line count. **Code is for people, not machines.**
- Rule of thumb: if a colleague has to ask you to explain it, it's too clever. Using a pattern because it's clever, not clear? Rewrite it plainly.

BAD

```
var r = orders.Where(o => o.S == "A")
  .Select(o => o.T * (1 - o.D))
  .ToList();
```

GOOD

```
var activeOrders = orders
  .Where(o => o.Status == ACTIVE_STATUS);
var totals = activeOrders
  .Select(o => o.Total * (1 - o.DiscountRate))
  .ToList();
```

6 House C# Guidelines: Naming, Casing & Async Names

- Bad naming wastes time on every read; **consistency** means you know what to expect (lists like `customerList`, booleans like `isX / hasX`). House standards make the codebase look like one piece of work and keep reviews on logic.

Construct	Casing	Example
Classes, interfaces, enums, structs	PascalCase	`OrderProcessor`
Public method / property	PascalCase	`GetUserData(), UserName`
Private method	camelCase	`validateInput()`
Private field	_camelCase	`_userName`
Local variable / parameter	camelCase	`orderTotal, customerName`
Constant	CONSTANT_CASE (shown in course)	`RETIREMENT_AGE`
Method returning Task / Task<T>	Add **Async** suffix (public or private)	`GetOrderAsync()`

- **Async suffix is a contract**: returns a Task, must be awaited. Without it, `var user = GetUser(42);` silently gives a `Task<User>`.
- Have both versions? Pair names: `Save` and `SaveAsync`. No exceptions to the rule.
- Method names match the action: value-returning start with `Get / Fetch / Retrieve`; actions with `Send / Save / Process`. Hard to name? It probably does too many things.
- Quick check: if you'd write `new` before it, use PascalCase; if you assign to it, camelCase.
- **Common mistake:** arguing naming style in PR comments. Enforce with tools instead (see next topic).

7 Automated Formatting: Three-Tool System

- **Manual enforcement in PRs** wastes time, is inconsistent, and frustrates people. **Let machines check style; humans check design, logic and trade-offs.**
- Arguing format in PR comments delays merges over preferences unrelated to logic. Automate once, never argue again.

Tool	Role	Notes
.editorconfig	The single rule book (indent, spacing, naming, braces)	Commit to repo root. Define **first**: both other tools read it.

Tool	Role	Notes
Roslyn analyzers	Live warnings in IDE as you type; also catch quality issues (unused vars, unsafe patterns)	Run in IDE and CI/CD. Check the file is loaded if no warnings show.
dotnet format	Auto-fixes the whole codebase to match rules	Run before commit or in CI. `--verify-no-changes` to check only.

SAMPLE .editorconfig

```
root = true

[*.cs]
indent_style = space
indent_size = 4
max_line_length = 120
```

- **Setup flow:** define rules → run `dotnet format` once on the codebase (commit it in a **separate PR**) → enable Roslyn in IDE → add checks to build pipeline so violations fail the build.
- Start with **4-6 rules**; ask the team after a week and tune strictness. Too strict chokes devs, too loose solves nothing.
- Misconception: running dotnet format before creating .editorconfig fails (no rules). No IDE warnings does not always mean setup is done.
- Behavior never changes, only formatting. Reviews then focus on design, performance and async safety.

8 PR Review Checklist: Code Smells

- Review does **two jobs**: maintain quality and share knowledge. Your role is to **help teammates improve**, not judge them.
- A **checklist** keeps reviews fair, consistent and thorough. Order: **readability** → **code smells** → **async** → **feedback**.
- **Code smell** = works but hints at a better design (not a bug). Static analyzers teach you to spot them; toggle rules (Naming Clarity, Method Complexity, Magic Values, Duplication) until you no longer need the tool.

Smell	What to look for
Vague names	`o, t, i, tx, Execute()`; rename to show intent
Magic values	`30` repeated for password expiry; extract `PasswordExpiryDays`
Long / mixed methods	One method validating, calculating and saving
Duplication	Same validation in 3 places
Comments explaining bad code	Rename or restructure instead

- Pattern recognition grows with practice: pause on anything that catches your eye, confirm against the checklist.
- Praise good patterns too; it shows what the team values.

9 Async Pitfalls

Pitfall	Problem	Fix
Missing await	Gets a `Task` not data; code continues early; errors escape try/catch	`await` every async call
Fire-and-forget	Task never tracked; failures unnoticed; also `async void` outside UI events	Await it. `_ = Task` only signals intent, still risky
Blocking (`.Result`, `.Wait()`)	Freezes thread; deadlocks, timeouts, unresponsive app	Use `await`, keep async all the way up the chain

BAD

```
var user = GetUserAsync().Result;
SendEmailAsync(email);
var u = GetUserFromDb(id); // Task!
```

GOOD

```
var user = await GetUserAsync();
await SendEmailAsync(email);
var u = await GetUserFromDbAsync(id);
```

Reviewer tip: In PR review, search for calls ending in **Async()** with no **await** in front; they are almost always a bug. Explain the consequence (throughput, deadlock, scalability) when you flag it.

10 Giving Constructive Feedback

Step	Do this	Example
1. Be specific	Name the line/method/pattern	"In `ProcessOrder` line 35, `order.Id` is used after a null check ends."
2. Explain why	Tie to a principle (SRP, clarity, defensive programming)	"One method should have one reason to change."
3. Suggest a fix	Offer a concrete path, not an order	"Try a new `ValidateDates` method called first."
4. Be respectful	Collaborative language	"I wonder if...", "One option is...", "This might be clearer as..."

NON-CONSTRUCTIVE

```
"Your naming is bad."
"This is blocking. Don't."
"Check for null here."
```

CONSTRUCTIVE

```
"Consider cachedUserName instead of temp;
 it tells the next reader what it holds."
"This blocks the thread. Use await to free it
 for other requests."
```

- Too crowded? Split into two short comments. Match code to the **checklist standard** so feedback targets the code, not the person.
- Don't debate formatting in PRs; `.editorconfig` + formatter handle it. If feedback conflicts, ask a lead for the "why".

11 Key Learnings & Week 3 Gate Challenge

Module	Core idea
Module 2: Readable C# (about you)	Clear names, small methods, guard clauses
Module 3: House standards (about the team)	Naming rules, automated tools, focused reviews

- **Two classic mistakes:** clever LINQ nobody can debug, and formatting debates in PR comments.
- **Gate task:** clean up ~100 lines of messy C# with unit tests as the safety net.

Gate check	What it means
Tests stay green	Run before and after; failing means you broke logic
Zero format warnings	Run Roslyn, fix every warning
Meets the rubric	Clear names, good structure, checklist satisfied

Final recap: Loop until all three pass, then submit. Clear names + small methods + guards + automation + kind reviews = code your team trusts.